Instructor setup — GitHub Classroom + hybrid autograding

This guide turns the term project into a shareable GitHub Classroom assignment with **hybrid autograding**: real automated grading on the testing assignments (5–6) and mechanical presence/build checks on the requirements/design assignments (1–4). Document content is still graded by rubric.

What's in this starter

```
README.md                          # student-facing
.github/workflows/classroom.yml # autograding workflow (recommended)
.github/workflows/ci.yml         # build + tests + JaCoCo + Checkstyle/PMD/SpotBugs (PR checks)
.github/classroom/autograding.json  # classic autograding config (optional)
scripts/check_files.py           # presence checks (Assignments 1-4)
scripts/check_coverage.py        # JaCoCo 90% gate (Assignments 5-6)
docs/ , templates/               # deliverable folders + blank templates
```

One-time setup

1. **Create the classroom.** Go to https://classroom.github.com, create an organization (or use an existing one), and create a **Classroom**. Import your student **roster** (upload a CSV of names; students link their GitHub accounts when they accept).
2. **Create the template repo.** Push this starter to a new repo in your org (e.g. `sysc3020-termproject-starter`) and mark it a **template repository** (Settings → check *Template repository*). Copy the `SERG-Delft/jpacman` project into the repo (it is **self-contained** — no companion libraries) and confirm `./gradlew build` succeeds locally first.
3. **Enable the autograder.** The `classroom.yml` workflow uses the `classroom-resources/autograding-*` actions (no extra secrets needed for the presence/build/coverage checks). Static analysis (Checkstyle/PMD/SpotBugs) is built into the Gradle build — no extra secret needed.

Create the assignment(s)

You have two workable models for a term project that lives in **one** team repo:

Model A — one group assignment for the term (simplest). - In Classroom, click **New assignment** → **Group assignment**. Give it a title, set the **team size** (2–3), choose **visibility** (private), under *Starter code* select **your template repository**, and (optionally) add the autograding tests. - **Where the link appears:** on the **final "Invite" step of the wizard**, GitHub Classroom shows an **invitation URL** that looks like `https://classroom.github.com/a/AbC123xy`. *That* is the link you share. You can re-open it any time from the assignment's page (the **"Copy invite link"** button at the top). - **When students get access:** a student opens the link → **Accepts the assignment** → creates a new team or joins an existing one → GitHub then **creates that team's repository from your template**. From that moment the team has its repo and the workflows start running. (Nobody gets a repo until they accept — the link is the entry point.) - The `classroom.yml` workflow runs on every push and posts a combined "mechanical health" score. At each deadline, open the latest workflow run for a team and read the checks relevant to that assignment (labelled `A1 ... ... A6 ...`).

Model B — one Classroom assignment per milestone (per-milestone scores). - Create six group assignments from the same template. Point students at the same repo each time (or let Classroom import the existing repo). Each assignment can carry only the autograding tests for that milestone — copy the relevant entries from `autograding.json` into that assignment's autograding settings.

Model A is recommended: it matches "one codebase carried through the term" and needs only one invite link.

Coverage scope (the same for every team)

Every team engineers the same **fixed project subset**, so coverage is measured on a **fixed set of core-logic packages — identical for all teams**, not on a per-team subsystem. The default scope is the game logic packages (`board,level,npc`), which excludes the `ui/sprite` (GUI) code that cannot be unit-tested. To change it, set an **Actions variable** `COVERAGE_SCOPE` (comma-separated package substrings, e.g. `board,level,npc,points`), or edit the argument in `classroom.yml`. Lower the threshold (default `0.90`) if full core coverage proves too demanding.

Because all teams cover the same system, authenticity is enforced by the per-student **design defense**, the **commit history**, and the **AI-usage disclosure** — not by distinct scope.

What is autograded vs. rubric-graded

Assignment	Autograded (mechanical/real)	Points*	Rubric-graded (human)
A1 Requirements	SRS/RTM/charter present	10	SRS content, use cases, NFRs
A2 Design (structure)	class/sequence `.puml` present	10	diagram correctness & consistency
A3 Design (behaviour/arch)	statecharts + ADRs present	10	statechart/architecture quality
A4 Design (patterns)	RFC present; **build compiles**	5 + 10	pattern correctness, metrics analysis
A5 Testing	**tests pass**	20	test design, coverage of requirements
A6 Testing	**coverage $\geq$ 90%**	25	defect analysis, defense
all	AI-usage note present	10	—

* Points are the autograder's mechanical "health" score (sums to 100), **not** the assignment grade. Grades follow `Evaluation_Guide_and_Rubric`.

Bundling the JPacman code (so the invite link gives working code)

JPacman is open-source and **self-contained** — no companion libraries. Just ship it in the template:

1. Copy the `SERG-Delft/jpacman` project into the template repo (on top of your classroom files).
2. Keep JPacman's `LICENSE` and add an attribution line to `README.md`.
3. Confirm `./gradlew build` succeeds and produces the JaCoCo report locally before publishing.

Because the code ships in the template, students who accept the invite get a repo that builds immediately with `./gradlew build` — nothing to install.

Sharing the invite link with students

1. Open your assignment in Classroom and click **Copy invite link** (top of the assignment page). The URL looks like `https://classroom.github.com/a/AbC123xy`.
2. **Post it** where students will see it — Brightspace announcement, the course page, or email. (Anyone with the link + on your roster can accept.)
3. **Student flow:** click link → sign in to GitHub → **Accept this assignment** → create a team or join a teammate's → their team repo is created from your template. They then clone it and start Assignment 1.

That single link is all students need — you do **not** create a repo for each team by hand; Classroom does it when they accept.

Tuning the checks

- Tighten presence globs in `scripts/check_files.py` calls (in `classroom.yml`) to match any naming you require.
- Adjust the coverage threshold in the `check_coverage.py` argument (default `0.90`).
- If teams keep tests in a separate module, point the `./gradlew` steps at it.